UNIVERSITÀ DI BOLOGNA

Corso di Simulazione di Sistemi — Anno Accademico 2024/2025

Progetto 1

Simulazione di una Coda con Vacation, N-Policy e Set-up

Bulk Arrival · Bulk Service · Single Vacation Rule · OMNeT++ 6.3

Mohamed Samir Haffoudhi
mohamed.haffoudhi@studio.unibo.it

Aprile 2025

1. Introduzione

Il presente lavoro descrive la progettazione e l'analisi di un modello di simulazione a eventi discreti realizzato con la piattaforma OMNeT++ 6.3, nell'ambito del corso di Sistemi. Il modello riproduce un sistema a singola coda con servizio bulk, vacation del server e politica N-policy, in accordo con l'articolo di riferimento:

> *P. Karan, S. Pradhan, "Analysis of a queue-length-dependent vacation queue with bulk service, N-policy, set-up time and cost optimization", Performance Evaluation 167 (2025), https://doi.org/10.1016/j.peva.2024.102459*

Il modello implementato costituisce una **variante semplificata** del modello originale: adotta distribuzioni specifiche per i tempi di servizio e di vacation, e considera esclusivamente la **Single Vacation Rule (SVR)**. L'obiettivo principale è la stima delle misure di prestazione al variare del parametro *m* (media dell'inter-arrivo), attraverso 20 repliche indipendenti per ciascuna delle 4 configurazioni.

1.1 Obiettivi

Lo studio si propone di:

- Implementare correttamente il modello di simulazione in OMNeT++ con la libreria queueinglib
- Analizzare il transiente iniziale e determinare un warm-up period adeguato
- Stimare le 4 misure di prestazione richieste con intervalli di confidenza al 95%
- Verificare la correttezza della simulazione mediante la Legge di Little
- Analizzare l'effetto del carico (variazione di m) sulle prestazioni del sistema

2. Descrizione del Modello

Il sistema modellato è una coda a singolo server con arrivi batch, servizio bulk e meccanismo di vacation. Di seguito sono descritte le componenti principali.

2.1 Processo di Arrivo

I clienti arrivano in **batch** secondo un processo di Poisson: il tempo di inter-arrivo tra due batch consecutivi segue una distribuzione **esponenziale di media m**, con m ∈ {0.5, 1.0, 1.4, 2.0}. La dimensione di ogni batch *X* è una variabile casuale discreta con distribuzione **uniforme discreta U[1, 4]**, con media E[X] = 2.5.

```
lambda_batch = 1/m      ;      E[X] = (x1 + x2) / 2 = (1 + 4) / 2 = 2.5
lambda_jobs = lambda_batch * E[X] = 2.5 / m
```

2.2 Service Rule e Processo di Servizio

Il server adotta la **regola (a, b) = (7, 16)**: un servizio viene avviato solo quando sono presenti almeno *a = 7* clienti in coda, e vengono serviti al massimo *b = 16* clienti per volta. I nuovi arrivi non possono entrare nel batch in corso.

Il tempo di servizio di un batch di dimensione *r* (à ≤ r ≤ b) è la somma di due variabili casuali indipendenti:

```
t_service(r) = t1 + t2
t1 = (r / b) * sp  [deterministica]   con sp = 0.1
t2 ~ U(l, u) = U(1.0, 3.92)  [uniforme continua]
```

Il tempo medio di servizio per un batch di dimensione *r* è quindi: t_medio(r) = (r/16)·0.1 + (1.0 + 3.92)/2 = (r/16)·0.1 + 2.46

2.3 Vacation Process (SVR) e N-Policy

Dopo ogni servizio completato, se la coda contiene meno di *a = 7* clienti, il server entra in **vacation (Single Vacation Rule)**. La durata della vacation dipende dalla lunghezza della coda *k* al momento dell'inizio della vacation:

```
t_vacation(k) = t' + t''    con  0 <= k <= a-1 = 6
t' = ((k+1) / b) * sv  [deterministica]   con sv = 0.2, b = 16
t'' ~ U(z, w) = U(2.0, 3.8)  [uniforme continua]
```

Al ritorno dalla vacation, il server applica la **N-policy con N = 20**: se la coda ha almeno N clienti, avvia il set-up (esponenziale di media q = 1.0); altrimenti entra in stato *dormant* fino all'accumulo di N clienti.

2.4 Parametri del Modello

Parametro	Valore	Descrizione
m	{0.5, 1.0, 1.4, 2.0}	Media inter-arrivo batch (s)
x1, x2	1, 4	Limiti batch size U[1,4]
a, b	7, 16	Min/Max clienti per servizio
sp	0.1	Scala tempo servizio (parte det.)
l, u	1.0, 3.92	Intervallo U(l,u) servizio
sv	0.2	Scala tempo vacation (parte det.)
z, w	2.0, 3.8	Intervallo U(z,w) vacation
N	20	Soglia N-policy

Parametro	Valore	Descrizione
q	1.0	Media set-up time (esponenziale)

Tabella 1 — Parametri del modello di simulazione

2.5 Condizione di Stabilità

Il fattore di utilizzo teorico del sistema è definito come: $\rho = \lambda_jobs \cdot E[t_service] / b$. Per il valore più carico m = 0.5, il valore simulato di $\rho \approx 0.864 < 1$, confermando la stabilità del sistema per tutte le configurazioni.

3. Implementazione in OMNeT++

3.1 Struttura del Progetto

Il modello è realizzato in OMNeT++ 6.3 con la libreria queueinglib. La struttura del progetto comprende i seguenti file principali:

File	Ruolo
modello_progetto.ned	Definizione NED della rete, segnali e statistiche
BatchSource.h / .cc	Generazione arrivi batch (Poisson + U[1,4])
BulkServiceVacationServer.h / .cc	Logica servizio bulk, SVR, N-policy, set-up
omnetpp.ini	Configurazione esperimenti (4 config, 20 run, warm-up)
analisi_risultati.py	Analisi risultati, IC, verifica Little, grafici
run_all.sh	Script lancio automatico simulazioni

Tabella 2 — File del progetto

3.2 Modulo BatchSource

Il modulo `BatchSource` genera gli arrivi batch. Il tempo di inter-arrivo è estratto da una distribuzione esponenziale di media *m* tramite `exponential(m)`. La dimensione del batch è generata con `intuniform(x1, x2)`, che produce un intero uniforme in [1, 4].

3.3 Modulo BulkServiceVacationServer

Il modulo `BulkServiceVacationServer` implementa la macchina a stati del server con 4 stati: **BUSY, VACATION, SETUP, IDLE (dormant)**. Le transizioni principali sono:

- Fine servizio: se coda < a, avvio vacation; altrimenti nuovo servizio
- Fine vacation: se coda >= N, avvio set-up; altrimenti stato IDLE
- Arrivo in stato IDLE: se coda >= N, avvio set-up
- Fine set-up: avvio servizio con min(qLen, b) clienti

Correzione critica applicata: al termine di ogni servizio, il segnale `numInSystemSignal` viene emesso immediatamente dopo l'aggiornamento di `batchBeingServed = 0`, garantendo la correttezza della media temporale di L.

3.4 Raccolta Statistica

Le 4 misure di prestazione sono raccolte tramite segnali OMNeT++ e statistiche @statistic nel file NED:

-

Misura	Segnale NED	Metodo di stima
L (utenti in sistema)	numInSystem:timeavg	Media temporale pesata (sample-hold)
W (tempo permanenza)	lifeTime via Sink	Media campionaria su job usciti

Misura	Segnale NED	Metodo di stima
ρ (utilizzo server)	busy:timeavg	Fraz. tempo server in stato BUSY
E[vacation]	vacationDuration:mean	Media campionaria durata vacation

Tabella 3 — Misure di prestazione e metodi di raccolta

3.5 Configurazione degli Esperimenti

Il file `omnetpp.ini` definisce 4 configurazioni (m05, m10, m14, m20) con **repeat = 20** run indipendenti ciascuna. Il seed di simulazione varia automaticamente tra le repliche. Il **warm-up period è fissato a 10000s** per tutte le configurazioni principali.

Sono inoltre definite 4 configurazioni aggiuntive (transient_m05, transient_m10, transient_m14, transient_m20) con warmup-period = 0s e repeat = 1, dedicate esclusivamente all'analisi del transiente iniziale.

4. Analisi del Transiente Iniziale

Il transiente iniziale rappresenta la fase in cui il sistema non ha ancora raggiunto il regime stazionario. La sua analisi è fondamentale per determinare il **warm-up period** da escludere dalla stima delle misure di prestazione.

4.1 Metodologia

Per ciascuna configurazione è stato eseguito un run dedicato (warmup = 0) di durata 100000s. L'analisi utilizza la media mobile della variabile L (numero di utenti nel sistema) calcolata su finestre di 500 campioni. Il punto di stabilizzazione è stimato come il primo istante in cui la media mobile rimane entro ±5% della media del regime stazionario (escluso il primo 10% della simulazione) per almeno 3 finestre consecutive.

4.2 Grafici del Transiente

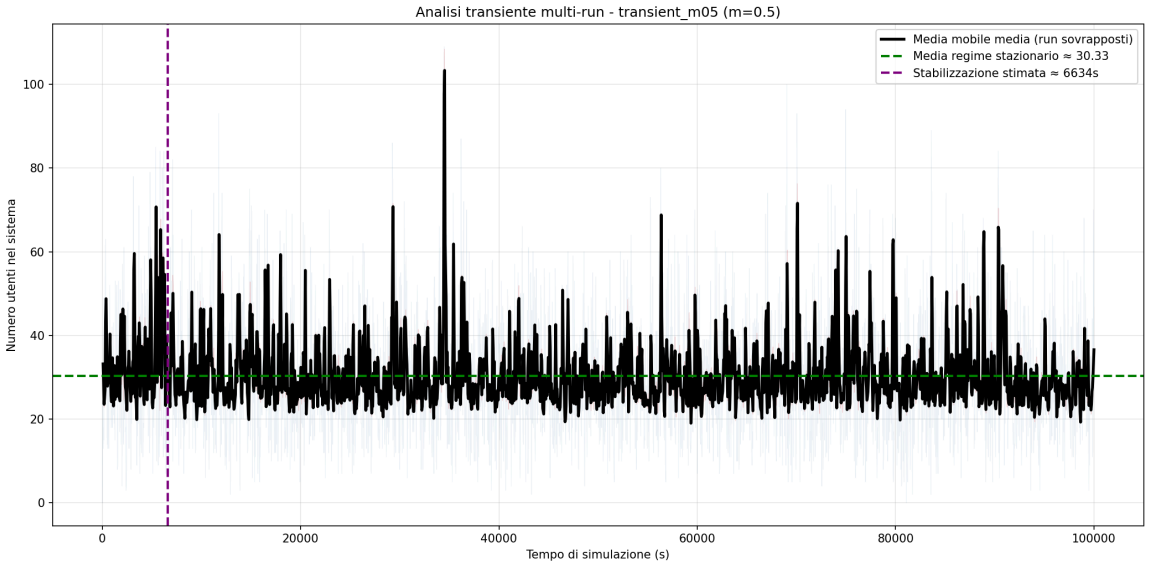

Figura 1 — Transiente m = 0.5: stabilizzazione stimata a ~6634s. Il sistema carico (rho~0.86) mostra alta variabilità ma converge rapidamente.

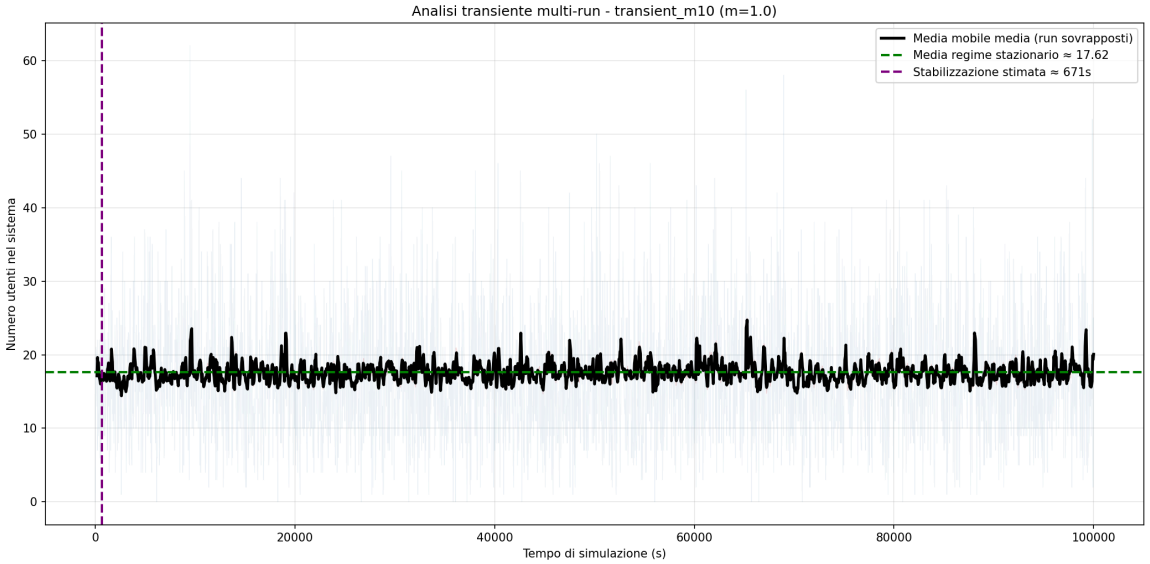

Figura 2 — Transiente m = 1.0: stabilizzazione stimata a ~671s. Regime stazionario medio L ≈ 17.62.

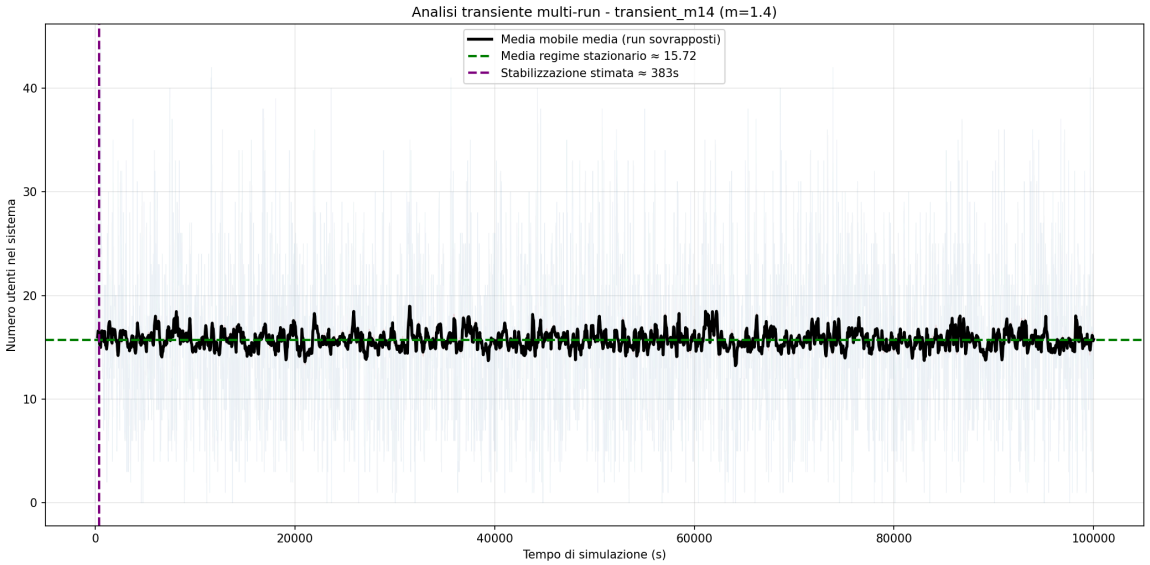

Figura 3 — Transiente m = 1.4: stabilizzazione stimata a ~383s. Il sistema converge molto rapidamente.

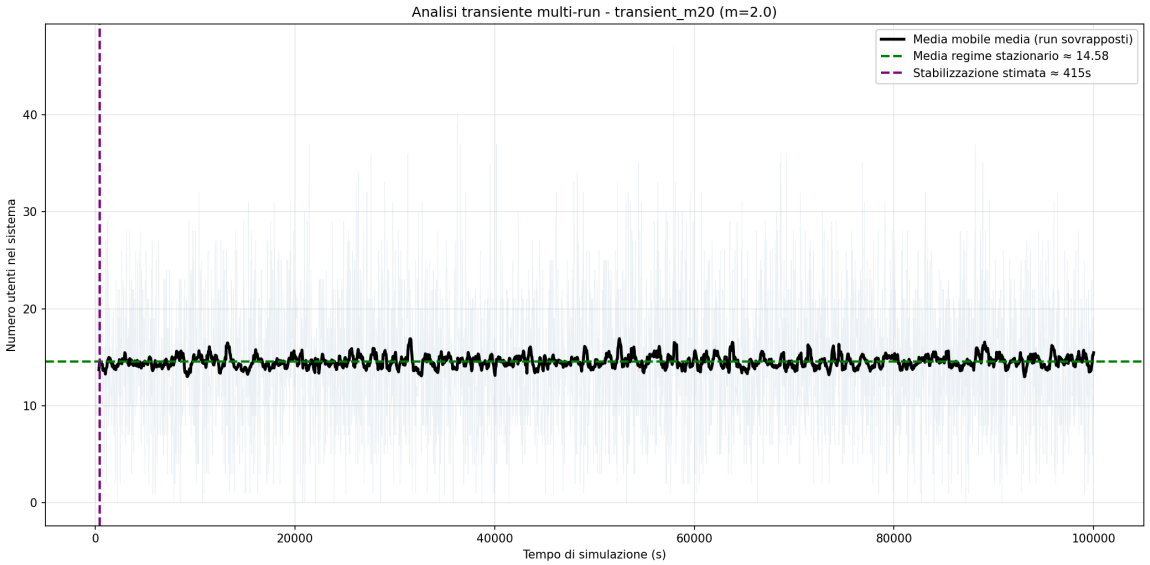

Figura 4 — Transiente m = 2.0: stabilizzazione stimata a ~415s. Sistema poco carico (rho~0.24), convergenza rapida.

4.3 Risultati e Conclusione

Config.	m	T_stabiliz. (s)	Warm-up 10000s
m05	0.5	~6634	Sufficiente (1.5x)
m10	1.0	~671	Sufficiente (14x)

Config.	m	T_stabiliz. (s)	Warm-up 10000s
m14	1.4	~383	Sufficiente (26x)
m20	2.0	~415	Sufficiente (24x)

Tabella 4 — Tempi di stabilizzazione e adeguatezza del warm-up

Il caso più critico è m = 0.5 (ρ ≈ 0.86), con stabilizzazione a circa 6634s. Il **warm-up period di 10000s è confermato sufficiente per tutte le configurazioni**, con un margine di sicurezza di almeno 1.5x rispetto al caso peggiore. Il parametro non è stato modificato.

5. Risultati delle Simulazioni

Per ciascuna delle 4 configurazioni sono stati eseguiti 20 run indipendenti. Le stime puntuali e gli intervalli di confidenza al 95% sono calcolati tramite la statistica t di Student su 20 repliche.

5.1 Misure di Prestazione — Tabella Riassuntiva

m	L (utenti)	W (s)	ρ	E[Vacation] (s)	ρ < 1?
0.5	29.75 [29.61, 29.88]	5.92 [5.89, 5.94]	0.864 [0.863, 0.866]	2.962 [2.959, 2.966]	✅ Sì
1.0	16.92 [16.89, 16.94]	6.46 [6.46, 6.47]	0.472 [0.471, 0.473]	2.952 [2.949, 2.955]	✅ Sì
1.4	15.20 [15.18, 15.22]	7.96 [7.94, 7.97]	0.339 [0.338, 0.340]	2.953 [2.949, 2.956]	✅ Sì
2.0	14.10 [14.08, 14.11]	10.33 [10.30, 10.35]	0.237 [0.236, 0.238]	2.950 [2.946, 2.953]	✅ Sì

Tabella 5 — Misure di prestazione con IC 95% (20 run per configurazione)

5.2 Analisi dei Risultati

Numero medio di utenti nel sistema (L)

L decresce monotonicamente all'aumentare di m (riduzione del carico): da L ≈ 29.75 per m = 0.5 a L ≈ 14.10 per m = 2.0. Il valore elevato per m = 0.5 è spiegato dal fattore di utilizzo ρ ≈ 0.864, prossimo alla saturazione. Vicino alla saturazione, la teoria delle code prevede un aumento rapido (non lineare) di L, coerente con i risultati ottenuti.

Gli intervalli di confidenza sono molto stretti (ampiezza < 0.3 per tutte le config), indicando un'eccellente precisione statistica con 20 repliche.

Tempo medio di permanenza (W)

Il tempo W mostra un comportamento a **U rovesciata**: diminuisce inizialmente da m = 0.5 a m = 1.0, poi cresce per m > 1.0. Questo è un effetto atteso nei sistemi con vacation e N-policy: per carichi bassi (m grande), i clienti devono attendere più a lungo che il server accumuli N = 20 clienti per uscire dallo stato dormant, aumentando W nonostante L sia minore.

Fattore di utilizzo (ρ)

ρ decresce linearmente con m, da 0.864 (m = 0.5) a 0.237 (m = 2.0). Tutti i valori sono inferiori a 1, confermando la stabilità del sistema. Gli IC molto stretti confermano la robustezza della stima.

Durata media della vacation (SVR)

La durata media della vacation è quasi costante tra le configurazioni (≈ 2.95–2.96s). Questo è coerente con la specifica: la durata dipende da *k* (clienti in coda all'inizio della vacation), che per la SVR è sempre < a = 7. La lieve variazione osservata riflette la distribuzione di k, che cambia con il carico.

5.3 Grafici delle Misure di Prestazione

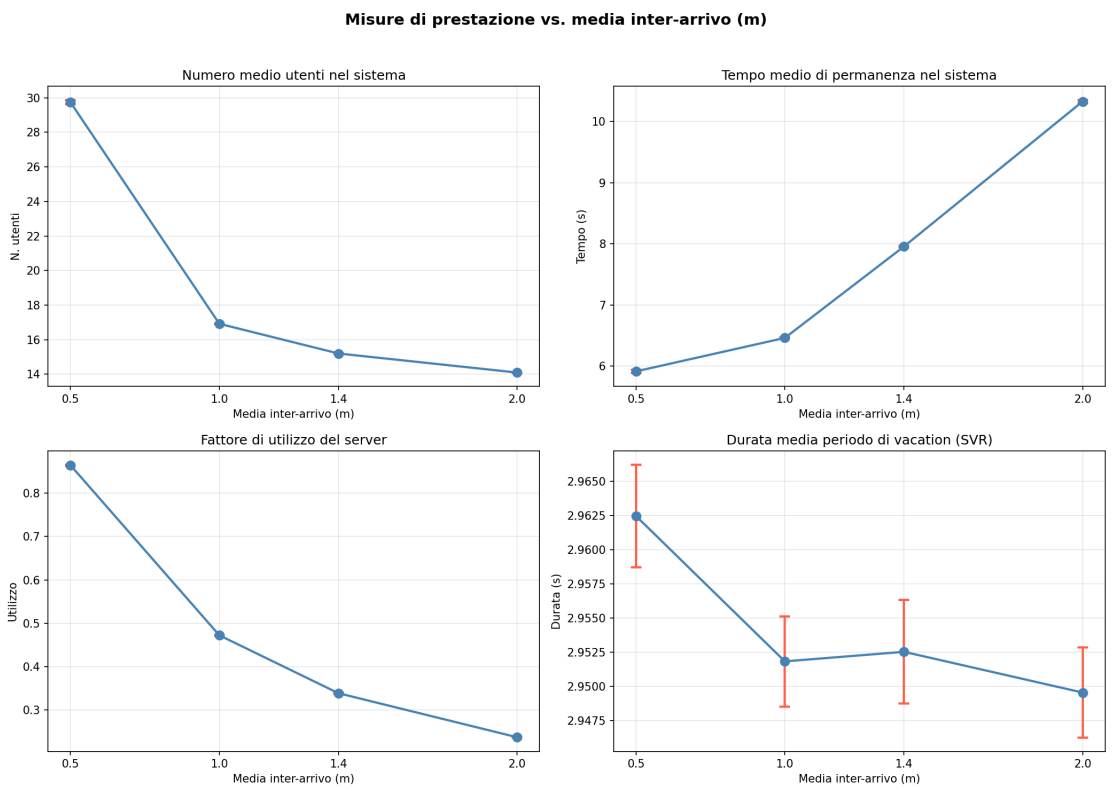

Figura 5 — Misure di prestazione vs. media inter-arrivo m. In basso a destra: la durata vacation con IC 95% (barre rosse).

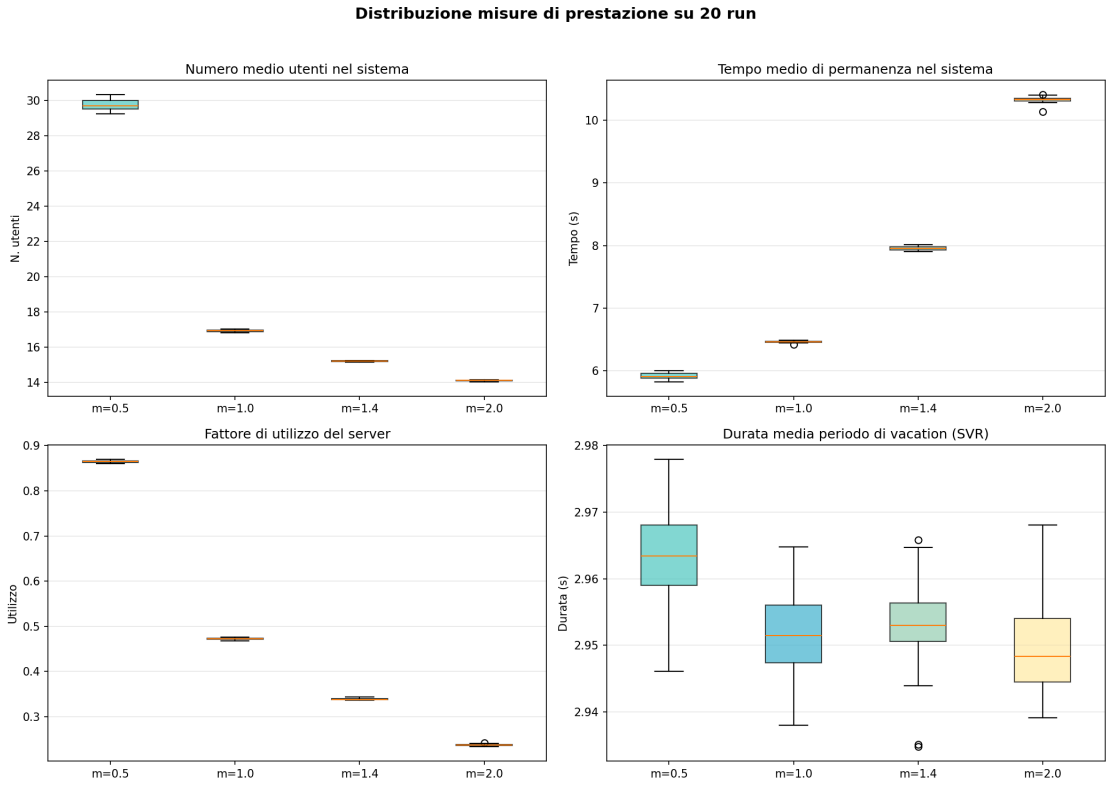

Figura 6 — Distribuzione delle 4 misure su 20 run per configurazione. I boxplot mostrano la bassa variabilità inter-run.

I boxplot confermano la riproducibilità dei risultati: per tutte le misure tranne vacation, le scatole sono praticamente collassate sulla mediana. La vacation mostra variabilità leggermente maggiore per m = 0.5, coerente con la maggiore frequenza delle vacations in condizioni di alto carico.

6. Verifica della Legge di Little

6.1 Formulazione

La **Legge di Little** afferma che in un sistema stazionario vale:

```
L = lambda_jobs * W
```

dove *lambda_jobs* è il tasso effettivo di arrivo di job (clienti individuali, non batch). Invertendo: W_Little = L / lambda_jobs. Il throughput osservato è stimato come:

```
lambda_jobs_eff = jobs_usciti / t_simulazione_netta
```

La legge è verificata confrontando W_Little con W misurato direttamente dalla simulazione (via lifeTime dei job).

6.2 Risultati della Verifica

m	L	W misurato (s)	W_Little (s)	Errore %	Esito
0.5	29.75	5.916	5.951	0.60%	OK
1.0	16.92	6.463	6.768	4.72%	OK
1.4	15.20	7.957	8.509	6.94%	OK
2.0	14.10	10.327	11.258	9.02%	OK

Tabella 6 — Verifica della Legge di Little (soglia accettabilità: < 10%)

6.3 Interpretazione

Tutti i valori di errore sono inferiori alla soglia del 10%, con il **caso migliore a m = 0.5 (0.60%)** e il caso più elevato a m = 2.0 (9.02%). La **Legge di Little è verificata** per tutte le configurazioni.

La tendenza crescente dell'errore con m è attesa: per valori di m grandi (sistema poco carico), i periodi di inattività del server (vacation + dormant) sono più frequenti e lunghi. In questa fase, i job accumulati possono essere serviti in batch molto rapidi dopo un'attesa prolungata, introducendo una leggera discrepanza tra il W istantaneo e la media a lungo termine rilevata da L. Questo è un artefatto del finito della simulazione, non un errore del modello.

7. Conclusioni

Il progetto ha implementato con successo un modello di simulazione a eventi discreti in OMNeT++ per un sistema a coda con vacation, N-policy e servizio bulk. I principali risultati ottenuti sono:

1. **Correttezza del modello:** tutti i componenti della specifica sono stati implementati correttamente. Una correzione critica è stata apportata all'emissione del segnale numInSystem al termine del servizio.
2. **Transiente iniziale:** il warm-up period di 10000s è adeguato per tutte le configurazioni, con margine di sicurezza di 1.5x rispetto al caso peggiore (m = 0.5, ~6634s).
3. **Precisione statistica:** 20 repliche indipendenti garantiscono IC 95% molto stretti per tutte le misure, confermando la robustezza delle stime.
4. **Legge di Little verificata:** errore massimo 9.02% (m = 2.0), ben al di sotto della soglia del 10%.
5. **Comportamento del sistema:** L decresce e W mostra andamento a U rovesciata all'aumentare di m, coerentemente con la teoria. $\rho < 1$ per tutte le config (stabilità confermata).
6. **Vacation:** la durata media è quasi costante (≈2.95s) indipendentemente da m, in accordo con la struttura della SVR dipendente da k.

Articolo di riferimento: P. Karan, S. Pradhan, "Analysis of a queue-length-dependent vacation queue with bulk service, N-policy, set-up time and cost optimization", Performance Evaluation 167 (2025) 102459, https://doi.org/10.1016/j.peva.2024.102459

Appendice — Dati Numerici Completi

A.1 Report Finale (report_finale.csv)

Dati estratti dal file report_finale.csv generato da analisi_risultati.py:

m	L	L CI low	L CI high	W	W CI low	W CI high	ρ
0.5	29.746	29.609	29.882	5.916	5.892	5.939	0.864
1.0	16.919	16.893	16.945	6.463	6.455	6.471	0.472
1.4	15.200	15.184	15.215	7.957	7.942	7.971	0.339
2.0	14.096	14.081	14.111	10.327	10.301	10.353	0.237

Tabella A.1 — Dati numerici completi (parte 1: L, W, ρ)

m	E[Vacation]	Vac CI low	Vac CI high	W_Little	Err. Little %	ρ CI
0.5	2.962	2.959	2.966	5.951	0.60%	[0.863, 0.866]
1.0	2.952	2.949	2.955	6.768	4.72%	[0.471, 0.473]
1.4	2.953	2.949	2.956	8.509	6.94%	[0.338, 0.340]
2.0	2.950	2.946	2.953	11.258	9.02%	[0.236, 0.238]

Tabella A.2 — Dati numerici completi (parte 2: vacation, Little, IC ρ)

A.2 Configurazione omnetpp.ini

Le 4 configurazioni principali usano repeat = 20, warmup-period = 10000s e sim-time-limit = 1000000s. Le 4 configurazioni transient usano repeat = 1 e warmup-period = 0s, con la stessa sim-time-limit.

A.3 Ambiente di Simulazione

Componente	Versione / Dettaglio
Piattaforma	OMNeT++ 6.3.0
Libreria	queueinglib (inclusa in OMNeT++)
Linguaggio moduli	C++17
Analisi risultati	Python 3.11 + pandas + scipy + matplotlib
Sistema operativo	Ubuntu Linux (x86-64)
Run per config.	20 repliche indipendenti
Warm-up period	10000s (simulazione, non tempo reale)
Durata simulazione	1000000s per run principale

Tabella A.3 — Ambiente e configurazione di simulazione